

Architecture & Trade-offs Document

account-ledger-core — Kazim Dafedar

This document covers architectural decisions, trade-offs, and production considerations that follow directly from the ledger implementation in this repository. It does not restate the event stream or Part 1's rule text; it assumes that context and reasons forward from it.

Append-only at scale

Nothing in this design has a database, an index, or a cache — it has three `ArrayList s` (`entries`, `holdEvents`, `errors`) inside `LedgerEngine`, and every read is a linear fold over one of them. `closingBalance` streams the entire `entries` list and filters by account, `valueDate`, and `postedDay` on every call. `activeHoldsTotal` and `currentHoldState` each walk the whole `holdEvents` list and reconstruct “latest status per authId” from scratch, every time, because there is no cached projection of current hold state anywhere. At 6 days and 10 events this is free. At 100× volume — call it 1,000 events across a handful of accounts, which is still a trivial workload for a real bank — every one of those reads becomes proportionally slower, and because holds are keyed by rebuilding a map over the *entire* history on every call, `activeHoldsTotal` degrades before `closingBalance` does: it's doing strictly more work per call (a map insert per event) for the same list length. The thing that breaks first is not “the database falls over,” because there isn't one — it's that authorization latency (`processAuthorization` calls both `closingBalance` and `activeHoldsTotal` synchronously, on the hot path of every hold request) grows linearly with the *total* lifetime volume of the account, forever, since nothing is ever evicted from the in-memory lists. That's the unbounded-state accumulation: true append-only, with zero deletion and zero snapshotting, means memory and per-read cost both grow without bound as a function of account age, not transaction rate at any given moment.

The cheapest structural fix that preserves the append-only/audit guarantee (rather than abandoning it for a database or a different consistency model) is a periodic immutable balance checkpoint per account: after each closed day, record a `(accountId, day, closingBalance, activeHoldsSnapshot)` row as its own appended, never-mutated record. `closingBalance` and `activeHoldsTotal` then only need to scan entries/holds with `postedDay > lastCheckpointDay`, not the full history — a bounded window instead of an unbounded one. This is a small, additive change: a new record type and a lookup at the start of each fold, no new dependency, no schema migration, no change to `process()`'s semantics. It does not fix everything, though, and I want to be explicit about that rather than oversell it. A retrospective query with `bucketDay` far in the past and `asOfDay` recent (the exact shape of criterion 1's Day-2-as-of-Day-5 query) still needs to scan back past any checkpoint taken *after*

`bucketDay`, because a checkpoint answers “as of the checkpoint day,” not “as of any earlier bucket.” And checkpoints introduce their own invalidation problem the current design doesn’t have: if a back-dated entry with `valueDate` before the last checkpoint’s day ever arrives (E7 is exactly this shape), any checkpoint taken after that value date is now stale for retrospective reads and must be recomputed or explicitly marked invalid — which is new bookkeeping this design has successfully avoided so far by just always scanning everything. Snapshotting trades read latency for a new class of correctness bug (silently stale checkpoints) that has to be designed for on purpose, not a free lunch.

Value-dated entries in production

Back-dated `value_date` is not a corner case here — E7 posts on Day 5 with `value_date` Day 2, and the design handles it correctly by construction. But “handled correctly” for a replay script and “handled correctly” for a UAE-licensed bank are different bars. The real operational surface is that this design lets an event change a *previously reported* accounting day’s true position without any notification mechanism at all. If Day 2’s closing balance was already extracted for an EOD position or liquidity report filed under CBUAE reporting obligations, and E7 lands three days later with `value_date = Day 2`, the retrospective query for Day 2 now returns a different number (−370.00 instead of 250.00) than what was actually reported at the time — silently, with nothing in this engine that flags “a previously-closed reporting day’s economic position just changed.” That is a real reconciliation and audit-trail gap, not a rounding nuance: a regulator reconciling this quarter’s filed EOD reports against the ledger’s current retrospective view would find a mismatch with no record explaining why, and no re-filing was ever triggered. The design’s honest defense today is only that the fee-assessment batch itself is never retroactively rerun — which protects fee/interest correctness, not regulatory-reporting correctness.

The one control I would add before going live is a hard backdating cutoff: once a `value_date` falls more than N days in the past relative to the current posting day (a bank-defined lookback window, plausibly aligned to the reporting cycle it needs to protect — e.g. matching whatever period is still open for amendment), an event that tries to book against that `value_date` cannot go through `bookUnconditional` as an ordinary entry at all. It must instead be routed through a distinct “late entry” event type that (a) books normally for ledger-balance purposes, exactly as today, but (b) unconditionally emits a separate, permanent audit record naming every already-filed report period it retroactively touches, and (c) requires a second, separately-approved acknowledgment before it’s considered settled for regulatory purposes — not because the ledger can’t book it, but because booking it silently is the actual regulatory exposure, not the arithmetic.

Authorization lifecycle

`HoldStatus` has four values, and only one of them — `SETTLED` — is a “matching settlement.” Every other way a hold in this model ends, plus the ones a production system needs that this model doesn’t have yet:

- **Declined at authorization time (`DECLINED`).** The real-world case is a hold request that the issuer refuses outright because it would drive available balance negative — no funds ever move, no active hold is ever created. This is not “approved then abandoned”; Auth-B in the scenario is exactly this case (the account was already overdrawn from E7 by the time Auth-B was requested), and the system behavior is that it’s booked as its own permanent `HoldEvent`, never a silent no-op.
- **Settlement against an unknown authorization (`REJECTED_UNKNOWN_AUTH`).** The real-world case is fraud, an integration bug, or a settlement message arriving for an authorization this ledger genuinely never saw (E6 / Auth-Z). The mandated behavior is: reject, move zero funds, and record an `ErrorRecord` — never book against an id you can’t independently verify was ever approved.
- **Expiry/timeout — genuinely missing today.** Card networks auto-expire uncaptured holds after a fixed window (commonly 7–30 days depending on merchant category). Auth-B, if it *had* been approved, would sit `APPROVED` forever in this design — there is no clock anywhere in `LedgerEngine`. The behavior I’d mandate: a scheduled sweep that, for any hold still `APPROVED` past N days, books its own immutable `EXPIRED HoldEvent` (a new enum value, not a reuse of `DECLINED` or `SETTLED`) and releases the hold from `activeHoldsTotal` — never silently dropped or inferred from absence.
- **Voluntary release/cancellation — also missing.** A merchant or cardholder can cancel an authorization before capture (e.g. a canceled order). Today the only way a hold’s active status ends is `SETTLED`, `DECLINED` -at-creation, or the missing expiry path above; there is no event type for “cancel this specific still-active hold.” I’d add a `ReleaseEvent` producing a `RELEASED HoldEvent`, distinct from `SETTLED` so a report can tell “captured” apart from “voluntarily let go.”
- **Partial settlement against a hold that isn’t fully resolved — also missing.** Today any settlement, partial or not, fully terminates the hold in one step (AMBIGUITIES.md item 6). Real card processors sometimes need multiple partial captures against one authorization (e.g. a hotel folio settled in installments) before the hold is exhausted. That needs a hold model with a remaining-amount field derived from a *sum* of settlements against one `authId`, not a single terminal state — a materially bigger change than the other three, which is exactly why it’s flagged here as cut rather than quietly handled.

What you cut and why

- **No concurrency control.** The whole engine is single-threaded, driven by one ordered replay. There is no per-account locking, optimistic version check, or serialization boundary anywhere in `LedgerEngine`. That's fine for a deterministic 6-day, 10-event replay; it is not fine for a production ledger where two events for the same account can arrive concurrently. The deferred risk is concrete: two threads calling `processAuthorization` for the same account at the same instant would both read the same pre-update `activeHoldsTotal / closingBalance` and could both approve holds that, taken together, drive available balance negative — the exact invariant the design exists to protect, broken by a race the single-threaded replay can never exercise.
- **No persistence, no crash recovery.** `entries`, `holdEvents`, and `errors` live in a JVM heap and nowhere else. Every guarantee in this document — append-only, auditable, reconstructible state — evaporates on process restart. A production version needs a write-ahead log or equivalent durable store before any of the “nothing is ever mutated or deleted” reasoning means anything outside of one process's lifetime.
- **No over-settlement validation.** `processSettlement` never checks `settleAmount` against the original hold amount — the one deliberately shipped gap, covered by the intentionally-failing `OverSettlementExceedsHoldFailingTest`. Flagged here specifically from a production-risk angle, not a testing one: as shipped, a settlement message can move more money out of an account than was ever actually held against it, with no rejection and no error record — a real path for erroneous or malicious over-capture to go completely unflagged.
- **No cross-currency / FX handling.** ACC-001 (AED) and ACC-002 (BHD) never interact anywhere in the engine — there is no conversion path, no FX rate input, no concept of a cross-currency transfer or settlement. A real bank needs FX conversion rules, rate sourcing, and almost certainly a different rounding/scale reconciliation story than the single-currency `Money.round` chokepoint this design relies on.
- **No idempotency or dedup on inbound events.** `process(Event)` trusts the caller never to submit the same event twice; there is no event-id dedup table anywhere. Replaying `E7` a second time would book it a second time, in full, with no detection. Production ingestion needs an idempotency key per event id checked before booking, which this design has no place to put yet because there's no persisted event log to check against.